

```

# =====
# CELL 1: ENVIRONMENT SETUP & MODULAR FILE GENERATION (FIXED SYNTAX)
# =====
import sys

print("📦 Installing deployment dependencies...")
!{sys.executable} -m pip install -q fastapi uvicorn streamlit transformers torch

# 1. Create requirements.txt
with open("requirements.txt", "w") as f:
    f.write("fastapi\nuvicorn\nstreamlit\ntransformers\ntorch\nrequests\nnest_asyncio\n")

# 2. Create models.py (Using raw strings to prevent string break typos)
with open("models.py", "w") as f:
    f.write(r'''# models.py - Model Initialization and Logic Engine
import torch
from transformers import pipeline

class TinyLlamaEngine:
    def __init__(self):
        self.persona_prompt = "<|system|>\nYou are an expert Python Code Review
        print("🚚 Loading local TinyLlama Pipeline...")
        self.generator = pipeline("text-generation", model="TinyLlama/TinyLlama-1.1B-Chat-v1.0")
        print("✅ Model loaded successfully.")

    def generate_response(self, user_prompt: str, max_tokens: int = 120) -> str:
        full_prompt = f"{self.persona_prompt}<|user|>\n{user_prompt}<|end|>\n<|assistant|>
        outputs = self.generator(full_prompt, max_new_tokens=max_tokens, do_sample=True)
        return outputs[0]["generated_text"].split("<|assistant|>\n")[-1].strip()

'''

# 3. Create main.py
with open("main.py", "w") as f:
    f.write(r'''# main.py - FastAPI Gateway Routing Application
import time
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from models import TinyLlamaEngine

app = FastAPI(title="LLM TinyLlama Production API Serving Endpoint")
engine = TinyLlamaEngine()

class InferenceRequest(BaseModel):
    prompt: str
    max_tokens: int = 120

class InferenceResponse(BaseModel):
    response: str
    latency_seconds: float

@app.post("/api/generate", response_model=InferenceResponse)
async def generate_endpoint(payload: InferenceRequest):
    if not payload.prompt.strip():
        raise HTTPException(status_code=400, detail="Prompt text cannot be empty")
    try:
        start_time = time.time()
        ai_response = engine.generate_response(payload.prompt, payload.max_tokens)
        return InferenceResponse(response=ai_response, latency_seconds=round(time.time() - start_time, 2))
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Internal server error: {str(e)}")
'''

```

```

        except Exception as e:
            raise HTTPException(status_code=500, detail=str(e))
    '''

# 4. Create client.py
with open("client.py", "w") as f:
    f.write(r'''# client.py - Streamlit Chat UI Presentation Interface
import streamlit as st
import requests

st.title("🤖 TinyLlama Python Code Reviewer")
BACKEND_URL = "http://localhost:8000/api/generate"

if "messages" not in st.session_state: st.session_state.messages = []
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]): st.markdown(msg["content"])

if user_input := st.chat_input("Paste your code here..."):
    with st.chat_message("user"): st.markdown(user_input)
    st.session_state.messages.append({"role": "user", "content": user_input})
    with st.chat_message("assistant"):
        try:
            res = requests.post(BACKEND_URL, json={"prompt": user_input, "max_
            output_text = res.json()["response"] if res.status_code == 200 else
        except: output_text = "Connection Error"
        st.markdown(output_text)
        st.session_state.messages.append({"role": "assistant", "content": output
    '''

print("✅ All files overwritten with clean formatting: models.py, main.py, cli

```

📦 Installing deployment dependencies...

✅ All files overwritten with clean formatting: models.py, main.py, clien

```

# =====
# CELL 2: IN-NOTEBOOK FASTAPI TESTING LOOP
# =====
import nest_asyncio
from fastapi.testclient import TestClient
from main import app

# Apply notebook async patch
nest_asyncio.apply()
client = TestClient(app)

print("\n🚀 Testing Local FastAPI Server Routing Architecture...")
test_prompt = "How do I optimize a basic Python for-loop?"
response = client.post("/api/generate", json={"prompt": test_prompt, "max_token

print("\n=== 📊 VERIFIED BACKEND METRICS OUT ===")
print(f"Status Code: {response.status_code}")
print(f"Server Response:\n{response.json()['response']}")
print(f"Latency Audit: {response.json()['latency_seconds']} seconds")

```

 Loading local TinyLlama Pipeline...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:11
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your setting
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to accept
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please see
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
config.json: 100% 608/608 [00:00<00:00, 39.1kB/s]
[transformers] `torch\_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100% 2.20G/2.20G [00:14<00:00, 187MB/s]
Loading weights: 100% 201/201 [00:18<00:00, 14.37it/s]
generation_config.json: 100% 124/124 [00:00<00:00, 8.23kB/s]
tokenizer_config.json: 100% 1.29k/1.29k [00:00<00:00, 49.5kB/s]
tokenizer.model: 100% 500k/500k [00:00<00:00, 1.31MB/s]
tokenizer.json: 100% 1.84M/1.84M [00:00<00:00, 29.1MB/s]
special_tokens_map.json: 100% 551/551 [00:00<00:00, 49.9kB/s]
[transformers] Passing `generation\_config` together with generation-relat
[transformers] Both `max\_new\_tokens` (=50) and `max\_length` (=2048) seem t
 Model loaded successfully.

 Testing Local FastAPI Server Routing Architecture...
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokeniz

===  VERIFIED BACKEND METRICS OUT ===

Status Code: 200

Server Response:

Sure, here are some tips to optimize a basic Python for-loop:

1. Use a loop variable: Instead of declaring a variable for each iteration
Latency Audit: 24.036 seconds

